



Universidad Nacional de San Antonio Abad del Cusco

Departamento Académico de Informática

Escuela Profesional: Ing. Informática y de Sistemas

COMPUTACIÓN GRÁFICA I

Práctica N° 07

TRANSFORMACIÓN 2D

ESCALA & ROTACIÓN

Estudiante: Efrain Vitorino Marin

Código: 160337

Profesor: Dr. Hans Harley Ccacyahuillca Bejar

Fecha: 1 de junio de 2026

1. Ejercicio 1: Escalamiento 2D

1.1. Descripcion

El programa implementa la transformación de **escalamiento homogéneo 2D** mediante matrices 3×3 en coordenadas homogéneas (column-major de OpenGL). Se definen tres variantes:

- **Escalamiento con factores independientes** (s_x, s_y):

$$S = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- **Escalamiento alrededor de un punto** (c_x, c_y): la transformación combinada $T(c_x, c_y) \cdot S(s_x, s_y) \cdot T(-c_x, -c_y)$ produce la matriz:

$$S_c = \begin{pmatrix} s_x & 0 & c_x(1 - s_x) \\ 0 & s_y & c_y(1 - s_y) \\ 0 & 0 & 1 \end{pmatrix}$$

- **Escalamiento uniforme**: caso particular con $s_x = s_y = s$.

Se renderiza un cuadrado centrado en el origen con una cruz interna. Los controles son: Q/A (escala X), W/S (escala Y), E/D (escala uniforme), R (reiniciar), ESC (salir).

1.2. Resultado en consola

```
=== CONTROLES ===
Q / A: Aumentar / Disminuir escala en X
W / S: Aumentar / Disminuir escala en Y
E / D: Aumentar / Disminuir escala uniforme
R: Reiniciar escalas
ESC: Salir
=====
Escala X: 1 | Escala Y: 1 | Escala Uniforme: 1
Escala X: 1 | Escala Y: 1 | Escala Uniforme: 1
```

1.3. Captura de pantalla

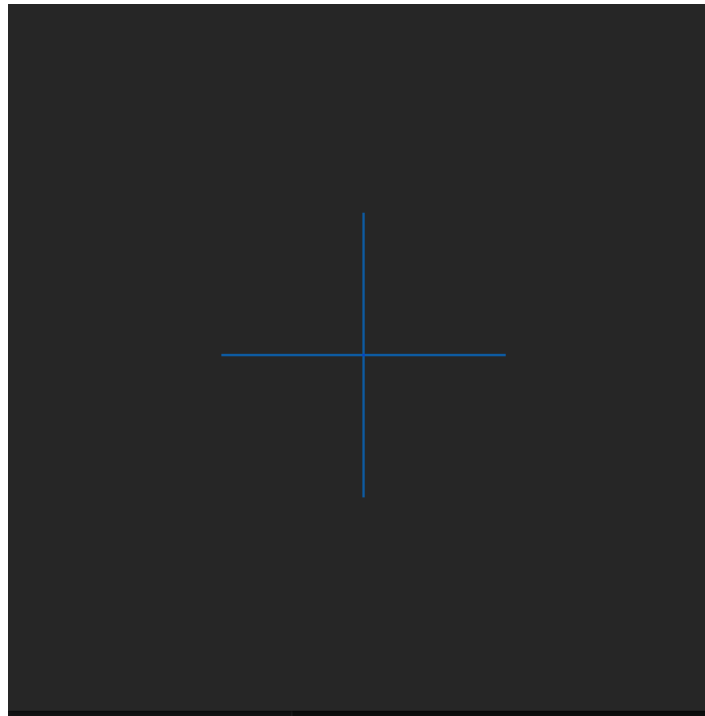


Figura 1: Ventana **escalamiento**: cuadrado con cruz de referencia (solo la cruz `GL_LINES` es visible en Core Profile).

2. Ejercicio 2: Rotación 2D

2.1. Descripción

El programa implementa la transformación de **rotación 2D** alrededor del origen y alrededor de un punto arbitrario (c_x, c_y) , usando matrices 3×3 .

- **Rotación alrededor del origen** (ángulo θ):

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- **Rotación alrededor de un punto** (c_x, c_y) : la transformación $T(c) \cdot R(\theta) \cdot T(-c)$ da:

$$R_c = \begin{pmatrix} \cos \theta & -\sin \theta & c_x - c_x \cos \theta + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y - c_x \sin \theta - c_y \cos \theta \\ 0 & 0 & 1 \end{pmatrix}$$

Se dibuja un triángulo en tres versiones: original (rojo), rotado alrededor del origen (verde), rotado a $1,5\theta$ alrededor del centroide (azul). Controles: R (girar +), T (girar -), ESC (salir).

2.2. Resultado en consola

```
Angulo de rotacion (verde): 0 grados
Angulo de rotacion (verde): 13 grados
Angulo de rotacion (verde): 25 grados
Angulo de rotacion (verde): 4 grados
Angulo de rotacion (verde): -22 grados
Angulo de rotacion (verde): -33 grados
```

2.3. Captura de pantalla

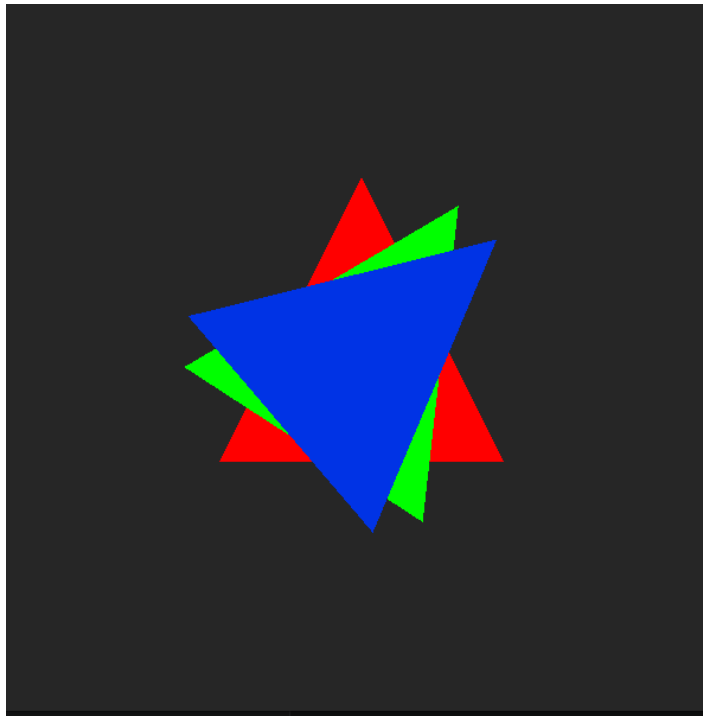


Figura 2: Ventana **rotacion**: triángulo original (rojo), rotado respecto al origen (verde) y rotado respecto al centroide a $1,5\theta$ (azul).

3. Actividad 1: Escalamiento de Círculo

3.1. Enunciado

“Escribir un programa que escale un círculo dos veces su dimensión original manteniendo fijo su centro.”

3.2. Descripción de la solución

Se genera un círculo usando `GL_TRIANGLE_FAN` con 64 segmentos, centrado en el origen con radio $r = 0,3$. El escalamiento $\times 2$ con centro fijo se obtiene con:

$$S_c(2, 2, 0, 0) = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Como el centro es el origen, la traslación compensatoria es nula y la matriz coincide con el escalamiento simple. El círculo blanco es el original y el verde es el escalado.

3.3. Código fuente

```

1  #include <GL/glew.h>
2  #include <GLFW/glfw3.h>
3  #include <glm/glm.hpp>
4  #include <glm/gtc/type_ptr.hpp>
5  #include <iostream>
6  #include <string>
7  #include <cmath>
8  #include <vector>
9
10 const char* vertSrc = R"(
11 #version 330 core
12 layout(location = 0) in vec2 aPos;
13 uniform mat3 uTransform;
14 void main() {
15     vec3 p = uTransform * vec3(aPos, 1.0);
16     gl_Position = vec4(p.xy, 0.0, 1.0);
17 }
18 )";
19
20 const char* fragSrc = R"(
21 #version 330 core
22 uniform vec4 uColor;
23 out vec4 fragColor;
24 void main() { fragColor = uColor; }
25 )";
26
27 GLuint shaderProg, vao, vbo;
28 int numVerticesCirculo = 0;
29 const float CENTRO_X = 0.0f, CENTRO_Y = 0.0f, RADIO = 0.3f;
30
31 void matrizIdentidad(float M[9]) {
32     M[0]=1; M[3]=0; M[6]=0;
33     M[1]=0; M[4]=1; M[7]=0;
34     M[2]=0; M[5]=0; M[8]=1;
35 }
36
37 void matrizEscalamientoAlrededorDe(float sx, float sy, float cx, float cy, float M[9]){
38     float tx = cx - sx*cx, ty = cy - sy*cy;
39     M[0]=sx; M[3]=0; M[6]=tx;
40     M[1]=0; M[4]=sy; M[7]=ty;
41     M[2]=0; M[5]=0; M[8]=1;
42 }
43
44 void inicializarGeometria() {

```

```

45     const int segmentos = 64;
46     std::vector<float> v;
47     v.push_back(CENTRO_X); v.push_back(CENTRO_Y);
48     for (int i = 0; i <= segmentos; i++) {
49         float a = 2.0f * M_PI * i / segmentos;
50         v.push_back(CENTRO_X + RADIO * cos(a));
51         v.push_back(CENTRO_Y + RADIO * sin(a));
52     }
53     numVerticesCirculo = (int)(v.size() / 2);
54     glGenVertexArrays(1, &vao); glGenBuffers(1, &vbo);
55     glBindVertexArray(vao);
56     glBindBuffer(GL_ARRAY_BUFFER, vbo);
57     glBufferData(GL_ARRAY_BUFFER, v.size()*sizeof(float), v.data(), GL_STATIC_DRAW);
58     glVertexAttribPointer(0,2, GL_FLOAT, GL_FALSE, 2*sizeof(float), (void*)0);
59     glEnableVertexAttribArray(0);
60     glBindVertexArray(0);
61 }
62
63 int main() {
64     glfwInit();
65     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
66     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
67     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
68     GLFWwindow* window = glfwCreateWindow(600,600,
69         "Actividad 1 - Escalamiento de circulo", NULL, NULL);
70     glfwMakeContextCurrent(window);
71     glewExperimental = GL_TRUE; glewInit();
72     glClearColor(0.15f,0.15f,0.15f,1.0f);
73     glEnable(GL_BLEND); glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
74     // compilarShaders(...) omitido por brevedad
75     inicializarGeometria();
76     GLint locMat = glGetUniformLocation(shaderProg,"uTransform");
77     GLint locColor = glGetUniformLocation(shaderProg,"uColor");
78     float M[9];
79     while (!glfwWindowShouldClose(window)) {
80         if (glfwGetKey(window, GLFW_KEY_ESCAPE)==GLFW_PRESS)
81             glfwSetWindowShouldClose(window,true);
82         glClear(GL_COLOR_BUFFER_BIT);
83         glUseProgram(shaderProg);
84         glBindVertexArray(vao);
85         // Circulo original (blanco)
86         matrizIdentidad(M);
87         glUniformMatrix3fv(locMat,1,GL_FALSE,M);
88         glUniform4f(locColor, 1.0f, 1.0f, 1.0f, 0.6f);
89         glDrawArrays(GL_TRIANGLE_FAN, 0, numVerticesCirculo);
90         // Circulo escalado x2, centro fijo (verde)
91         matrizEscalamientoAlrededorDe(2.0f,2.0f,CENTRO_X,CENTRO_Y,M);
92         glUniformMatrix3fv(locMat,1,GL_FALSE,M);
93         glUniform4f(locColor, 0.0f, 1.0f, 0.0f, 0.5f);
94         glDrawArrays(GL_TRIANGLE_FAN, 0, numVerticesCirculo);
95         glBindVertexArray(0);
96         glfwSwapBuffers(window); glfwPollEvents();
97     }
98     glDeleteVertexArrays(1,&vao); glDeleteBuffers(1,&vbo);
99     glDeleteProgram(shaderProg);
100    glfwDestroyWindow(window); glfwTerminate();
101    return 0;
102 }

```

Listing 1: actividadCirculo.cpp

3.4. Resultado en consola

=== Actividad 1: Escalar circulo x2 manteniendo centro fijo ===

Blanco: circulo original (radio 0.3)

Verde: circulo escalado x2 con centro fijo en (0, 0)

ESC: Salir

3.5. Captura de pantalla

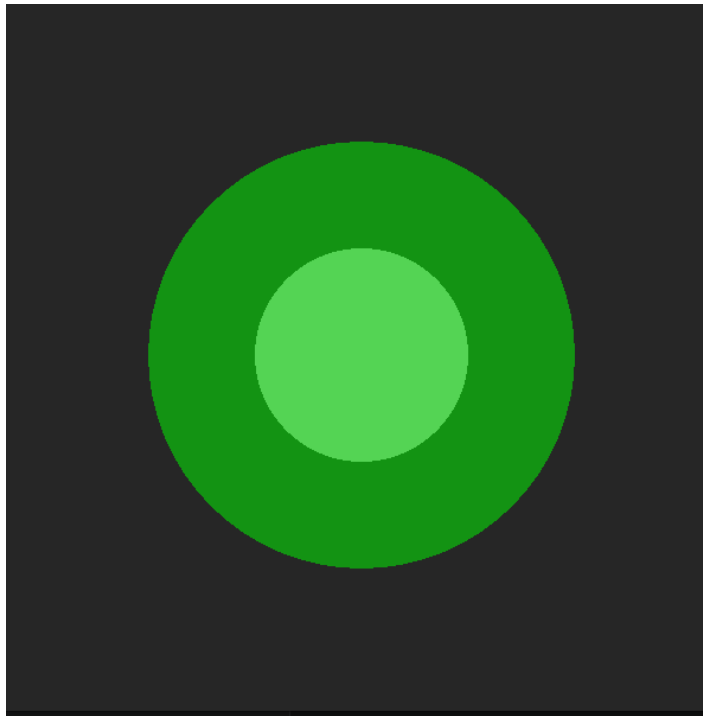


Figura 3: Actividad 1: círculo blanco (original, radio = 0,3) y círculo verde escalado $\times 2$ (radio = 0,6) con centro fijo en el origen.

4. Actividad 2: Traslación y Rotación de Rectángulo

4.1. Enunciado

“Realizar una traslación y luego una rotación de un rectángulo.”

4.2. Descripción de la solución

Se define un rectángulo centrado en el origen ($0,6 \times 0,3$ unidades). La transformación combina:

1. **Traslación** $T(0,4, 0,2)$
2. **Rotación** $R(\theta)$ alrededor del origen

La matriz combinada es $M = R(\theta) \cdot T(0,4, 0,2)$, obtenida con la función `multiplicarMatrices(R, T, TR)`. Al aplicarse de derecha a izquierda, primero se traslada el rectángulo y luego se rota el resultado.

La fórmula matemática es:

$$M = R(\theta) \cdot T = \begin{pmatrix} \cos \theta & -\sin \theta & 0,4 \cos \theta - 0,2 \sin \theta \\ \sin \theta & \cos \theta & 0,4 \sin \theta + 0,2 \cos \theta \\ 0 & 0 & 1 \end{pmatrix}$$

Controles: R (girar +), T (girar -), ESC (salir).

4.3. Código fuente

```

1  #include <GL/glew.h>
2  #include <GLFW/glfw3.h>
3  #include <glm/glm.hpp>
4  #include <glm/gtc/type_ptr.hpp>
5  #include <iostream>
6  #include <string>
7  #include <cmath>
8
9  const char* vertSrc = R"(
10 #version 330 core
11 layout(location = 0) in vec2 aPos;
12 uniform mat3 uTransform;
13 void main() {
14     vec3 p = uTransform * vec3(aPos, 1.0);
15     gl_Position = vec4(p.xy, 0.0, 1.0);
16 }
17 )";
18 const char* fragSrc = R"(
19 #version 330 core
20 uniform vec4 uColor;
21 out vec4 fragColor;
22 void main() { fragColor = uColor; }
23 )";
24
25 GLuint shaderProg, vao, vbo;
26
27 void matrizIdentidad(float M[9]) {
28     M[0]=1;M[3]=0;M[6]=0; M[1]=0;M[4]=1;M[7]=0; M[2]=0;M[5]=0;M[8]=1;
29 }
30 void matrizTraslacion(float tx, float ty, float M[9]) {
31     M[0]=1; M[3]=0; M[6]=tx;
32     M[1]=0; M[4]=1; M[7]=ty;
33     M[2]=0; M[5]=0; M[8]=1;
34 }
35 void matrizRotacion(float anguloGrados, float M[9]) {

```

```

36     float rad = anguloGrados * M_PI / 180.0f;
37     float c = cos(rad), s = sin(rad);
38     M[0]= c; M[3]=-s; M[6]=0;
39     M[1]= s; M[4]= c; M[7]=0;
40     M[2]=0; M[5]=0; M[8]=1;
41 }
42 // C = A * B (aplica B primero, luego A)
43 void multiplicarMatrices(const float A[9], const float B[9], float C[9]) {
44     for (int col=0; col<3; col++)
45         for (int f=0; f<3; f++)
46             C[col*3+f] = A[0*3+f]*B[col*3+0]
47                     + A[1*3+f]*B[col*3+1]
48                     + A[2*3+f]*B[col*3+2];
49 }
50
51 void inicializarGeometria() {
52     float v[] = {
53         -0.3f, 0.15f, 0.3f, 0.15f, 0.3f, -0.15f,
54         -0.3f, 0.15f, 0.3f, -0.15f, -0.3f, -0.15f
55     };
56     glGenVertexArrays(1,&vao); glGenBuffers(1,&vbo);
57     glBindVertexArray(vao);
58     glBindBuffer(GL_ARRAY_BUFFER,vbo);
59     glBufferData(GL_ARRAY_BUFFER,sizeof(v),v,GL_STATIC_DRAW);
60     glVertexAttribPointer(0,2,GL_FLOAT,GL_FALSE,2*sizeof(float),(void*)0);
61     glEnableVertexAttribArray(0);
62     glBindVertexArray(0);
63 }
64
65 int main() {
66     glfwInit();
67     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR,3);
68     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR,3);
69     glfwWindowHint(GLFW_OPENGL_PROFILE,GLFW_OPENGL_CORE_PROFILE);
70     GLFWwindow* window = glfwCreateWindow(600,600,
71     "Actividad 2 - Traslacion y Rotacion de rectangulo",NULL,NULL);
72     glfwMakeContextCurrent(window);
73     glewExperimental=GL_TRUE; glewInit();
74     glClearColor(0.15f,0.15f,0.15f,1.0f);
75     glEnable(GL_BLEND); glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
76     // compilarShaders(...) omitido por brevedad
77     inicializarGeometria();
78     GLint locMat = glGetUniformLocation(shaderProg,"uTransform");
79     GLint locColor = glGetUniformLocation(shaderProg,"uColor");
80     float M[9], T[9], R[9], TR[9];
81     float angulo = 0.0f;
82     const float TX=0.4f, TY=0.2f;
83     while (!glfwWindowShouldClose(window)) {
84         if (glfwGetKey(window,GLFW_KEY_ESCAPE)==GLFW_PRESS)
85             glfwSetWindowShouldClose(window,true);
86         if (glfwGetKey(window,GLFW_KEY_R)==GLFW_PRESS) angulo += 1.0f;
87         if (glfwGetKey(window,GLFW_KEY_T)==GLFW_PRESS) angulo -= 1.0f;
88         glClear(GL_COLOR_BUFFER_BIT);
89         glUseProgram(shaderProg);
90         glBindVertexArray(vao);
91         // Rectangulo original (rojo)
92         matrizIdentidad(M);
93         glUniformMatrix3fv(locMat,1,GL_FALSE,M);
94         glUniform4f(locColor,1,0,0,0.6f);
95         glDrawArrays(GL_TRIANGLES,0,6);
96         // Rectangulo trasladado + rotado (verde)
97         // M = R * T => aplica T primero, luego R
98         matrizTraslacion(TX,TY,T);
99         matrizRotacion(angulo,R);
100        multiplicarMatrices(R,T,TR);
101        glUniformMatrix3fv(locMat,1,GL_FALSE,TR);
102        glUniform4f(locColor,0,1,0,0.7f);
103        glDrawArrays(GL_TRIANGLES,0,6);
104        glBindVertexArray(0);
105        glfwSwapBuffers(window); glfwPollEvents();
106    }
107    glDeleteVertexArrays(1,&vao); glDeleteBuffers(1,&vbo);
108    glDeleteProgram(shaderProg);

```

```
109     glfwDestroyWindow(window); glfwTerminate();  
110     return 0;  
111 }
```

Listing 2: actividadRectangulo.cpp

4.4. Resultado en consola

```
=== Actividad 2: Traslacion + Rotacion de rectangulo ===  
Rojo:  rectangulo original en el origen  
Verde: rectangulo trasladado (0.4, 0.2) y luego rotado  
R: Aumentar angulo | T: Disminuir angulo | ESC: Salir  
Angulo de rotacion: 0 grados  
Angulo de rotacion: 12 grados  
Angulo de rotacion: 28 grados  
Angulo de rotacion: -20 grados  
Angulo de rotacion: -80 grados  
Angulo de rotacion: -140 grados  
Angulo de rotacion: -144 grados  
Angulo de rotacion: -119 grados
```

4.5. Captura de pantalla

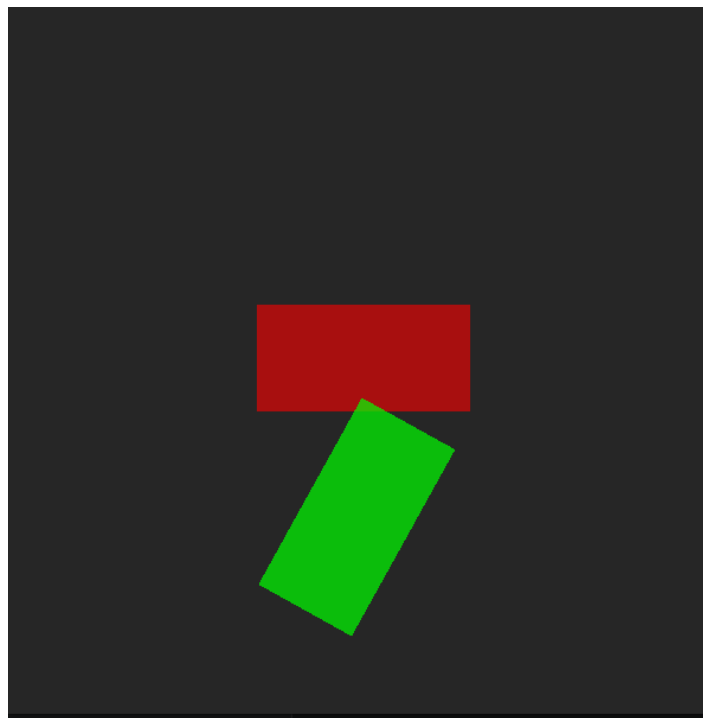


Figura 4: Actividad 2: rectángulo rojo (posición original en el origen) y rectángulo verde (trasladado a (0,4, 0,2) y rotado $\theta \approx -119^\circ$).